```
import pandas as pd
import numpy as np

# 1. Dataset ko load kiya
df = pd.read_csv("/content/APL_Logistics.csv" , encoding="ISO-8859-1")

# 2. Data ka basic shape aur columns check kiya
print("Total Rows and Columns:", df.shape)
print("\nAll Column Names:\n", df.columns.tolist())

# 3. Pehle 5 records ka preview dekhne ke liye
df.head()
```

```
Total Rows and Columns: (180519, 40)

All Column Names:
 ['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit per order', 'Sales per customer', 'Delivery
```

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk	Category Id	Category Name	Customer City	...	Orc It Quanti
0	DEBIT	6	4	159.69	472.45	Late delivery	1	9	Cardio Equipment	Brownsville	...	
1	DEBIT	4	4	48.71	167.96	Shipping on time	0	29	Shop By Sport	Littleton	...	
2	DEBIT	4	4	87.36	181.99	Shipping on time	0	48	Water Sports	Littleton	...	
3	DEBIT	6	4	-41.89	175.99	Late delivery	1	48	Water Sports	Littleton	...	
4	DEBIT	6	4	10.00	40.00	Late delivery	1	24	Women's Apparel	Littleton	...	

5 rows × 40 columns

This cell loads the APL Logistics dataset from a CSV file, prints its total number of rows and columns, lists all column names, and displays the first 5 rows to provide a quick preview of the data.

```
df.shape
```

```
(180519, 40)
```

This cell simply prints the shape (number of rows and columns) of the DataFrame `df`, confirming the dimensions of the loaded data.

```
import matplotlib.pyplot as plt
import seaborn as sns

# ======================================================
# STEP 1: DATA CLEANING & BLANK COLUMN CHECK
# ======================================================
print("============ 1. MISSING / BLANK VALUES CHECK ============")
missing_data = df.isnull().sum()
missing_cols = missing_data[missing_data > 0]

if len(missing_cols) == 0:
    print("Kisi bhi column me koi blank value nahi hai.\n")
else:
    print("In columns me blank values mili hain:")
    print(missing_cols, "\n")

# Customer Lname me agar blank hai toh use 'Unknown' se fill karte hain
if 'Customer Lname' in df.columns:
    df['Customer Lname'] = df['Customer Lname'].fillna('Unknown')
```

```
# Inconsistent Rows ki Safai (Shipping days negative nahi ho sakte)
df = df[df['Days for shipping (real)'] >= 0]
```

```
============ 1. MISSING / BLANK VALUES CHECK ============
In columns me blank values mili hain:
Customer Lname      8
Customer Zipcode    3
dtype: int64
```

This cell performs an initial check for missing values in the DataFrame. It identifies columns with blank values, prints them, and specifically fills missing 'Customer Lname' values with 'Unknown'. It also filters out rows where 'Days for shipping (real)' is negative, as shipping days cannot be negative.

```
# 1. Customer Lname ki khali jagah par 'Unknown' likha
df['Customer Lname'] = df['Customer Lname'].fillna('Unknown')

# 2. Customer Zipcode ki khali jagah par sabse zyada aane wala zipcode (Mode) bhara
zip_mode = df['Customer Zipcode'].mode()[0]
df['Customer Zipcode'] = df['Customer Zipcode'].fillna(zip_mode)

# 3. Check kiya ki ab koi blank bacha toh nahi?
print("Remaining Blank Values in Dataset:", df.isnull().sum().sum())
```

```
Remaining Blank Values in Dataset: 0
```

This cell continues the data cleaning process by filling missing values. It replaces blank 'Customer Lname' entries with 'Unknown' and fills missing 'Customer Zipcode' entries with the most frequent zipcode (mode). Finally, it confirms that no blank values remain in the entire dataset.

Shipping Days ka Analysis: Code check kar raha hai ki orders ko sach me deliver hone me kitne din lag rahe hain (Real) aur system ne kitne din plan kiye the (Scheduled).

Reality vs Plan: Data se pata chal raha hai ki actual delivery ka average 3.50 din hai, jabki system standardly 2.93 din hi plan kar raha hai.

Delivery Gap: Zyadatar orders plan se late deliver ho rahe hain (jaise reality me 5-6 din lag jana), jiski wajah se logistics me delay (late delivery risk) aa raha hai.

Purpose: Yeh script logistics team ko un bottle-necks aur system gaps ko pehchane me madad karti hai jahan shipping process ko behtar karne ki zaroorat hai.

```
# Sirf in do main columns ka statistical summary check karte hain
for col in ['Days for shipping (real)', 'Days for shipment (scheduled)']:
    print(f"--- Statistics for: {col} ---")
    print(f"Mean (Average) : {df[col].mean():.2f}")
    print(f"Median (Middle): {df[col].median():.2f}")
    print(f"Mode (Frequent): {df[col].mode()[0]:.2f}")
    print("-" * 35)
```

```
--- Statistics for: Days for shipping (real) ---
Mean (Average) : 3.50
Median (Middle): 3.00
Mode (Frequent): 2.00
-----------------------------------
--- Statistics for: Days for shipment (scheduled) ---
Mean (Average) : 2.93
Median (Middle): 4.00
Mode (Frequent): 4.00
-----------------------------------
```

This cell calculates and prints descriptive statistics (mean, median, mode) for two key columns: 'Days for shipping (real)' and 'Days for shipment (scheduled)'. This helps in understanding the central tendency and most frequent values for actual and planned shipping times.

Real Delivery ka Pattern: Yeh graph dikhata hai ki APL Logistics ke zyadatar orders reality me 2 se 3 din ke andar deliver ho rahe hain (jo purple aur green lines se saaf dikh raha hai).

Late Deliveries ka Risk: Graph me right side ek lambi tail (hissa) dikh rahi hai, jiska matlab hai ki bohot se orders 5 ya 6 din tak khinch rahe hain, jo ki system ke planned schedule se kaafi zyada hai.

Main Takeaway: Yeh visual saaf batata hai ki actual delivery ka average (Mean = 3.50 din) system ke planned schedule se bada hai, jo logistics me delay aur late delivery ke risk ko highlight karta hai.

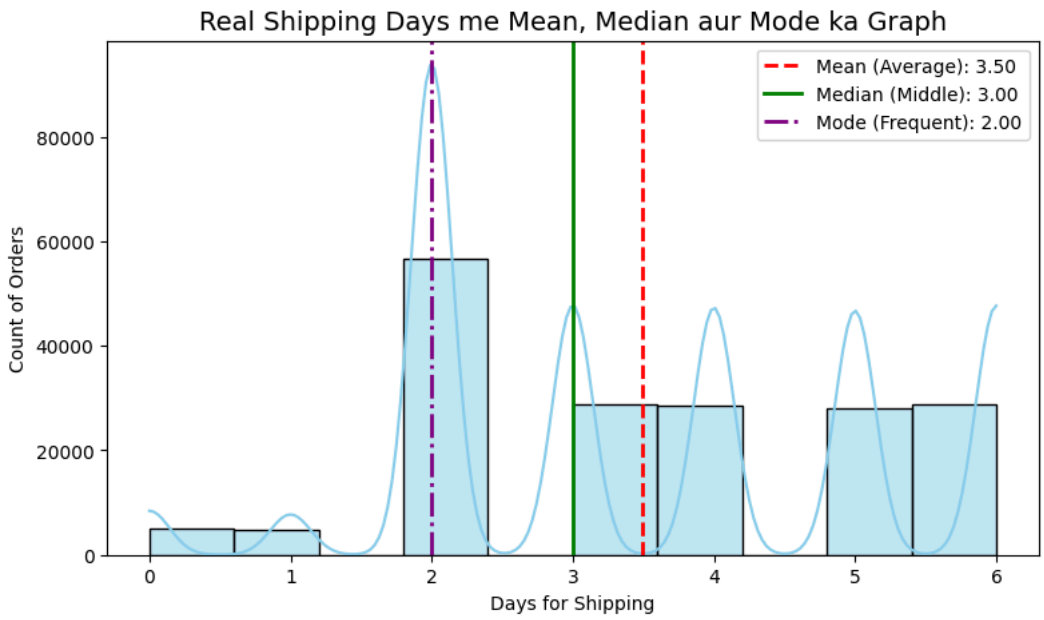
```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# 1. Real Shipping Days ka Data lekar values nikalna
data = df['Days for shipping (real)']
mean_val = data.mean()
median_val = data.median()
mode_val = data.mode()[0]

# 2. Graph (Distribution Plot) banana
plt.figure(figsize=(9, 5))
sns.histplot(data, kde=True, color='skyblue', bins=10)

# 3. Mean, Median, Mode ki lines khinchna
plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Mean (Average): {mean_val:.2f}')
plt.axvline(median_val, color='green', linestyle='-', linewidth=2, label=f'Median (Middle): {median_val:.2f}')
plt.axvline(mode_val, color='purple', linestyle='-.', linewidth=2, label=f'Mode (Frequent): {mode_val:.2f}')

# 4. Graph banana
plt.title('Real Shipping Days me Mean, Median aur Mode ka Graph', fontsize=14)
plt.xlabel('Days for Shipping')
plt.ylabel('Count of Orders')
plt.legend() # Isse top-right me pata chalega konsi line kiski hai
plt.show()
```



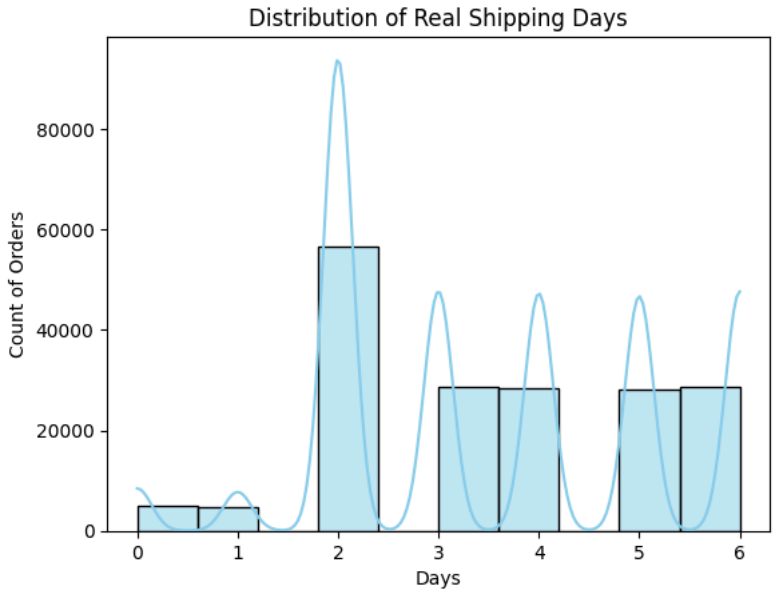
This cell generates a histogram to visualize the distribution of 'Days for shipping (real)'. It also overlays lines indicating the mean, median, and mode of these values, providing a clear graphical representation of the actual delivery time patterns.

Real shipping days distribution graph se pata chalta hai ki reality mein company ke zyadatar orders 2 se 3 din ke andar deliver ho rahe hain. Lekin isme ek lamba tail-end bhi dikh raha hai jahan bohot se orders ko deliver hone mein 5 ya 6 din lag rahe hain. Jab hum iski tulna scheduled timeline se karte hain, toh saaf pata chalta hai ki yahi woh orders hain jo delay ka shikar ho rahe hain. Yeh graph logistics team ko un bottle-necks aur regions ko pehchane mein madad karta hai jahan shipping process bohot dheemi chal rahi hai.

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Real Shipping Days (Actual Delivery) ka histogram plot
sns.histplot(df['Days for shipping (real)'], bins=10, kde=True, color='skyblue')
plt.title('Distribution of Real Shipping Days')
plt.xlabel('Days')
plt.ylabel('Count of Orders')

# Graph ko save karne ke liye (Colab ke files side-bar me save ho jayega)
plt.savefig('real_shipping_days_distribution.png')
```



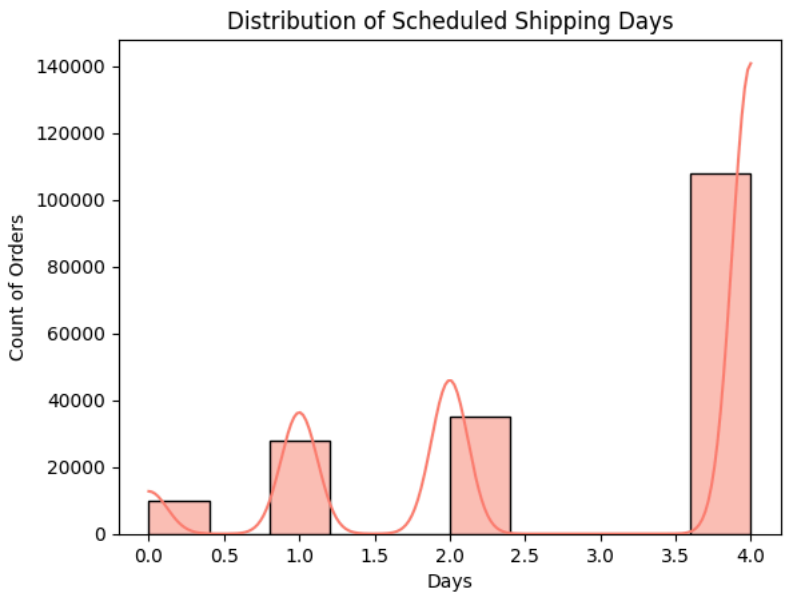
This cell creates a histogram plot for the 'Days for shipping (real)' column to visualize its distribution. It also saves this plot as a PNG image file for later reference.

Scheduled distribution graph se pata chalta hai ki company ka system standardly zyadatar orders ke liye 4 din ka delivery time plan karta hai. Jab hum iski tulna real shipping days wale graph se karte hain, toh saaf dikhta hai ki reality mein orders is timeline ko cross karke 5 ya 6 din tak khinch rahe hain. System ke isi planned schedule aur actual delivery ke beech ka farq hamare dataset mein delivery gap paida kar raha hai. Yeh gap hi naye orders ke liye late delivery ka risk badhata hai jise dashboard par monitor karna zaroori hai.

```
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Scheduled Shipping Days (Planned Delivery) ka histogram plot
sns.histplot(df['Days for shipment (scheduled)'], bins=10, kde=True, color='salmon')
plt.title('Distribution of Scheduled Shipping Days')
plt.xlabel('Days')
plt.ylabel('Count of Orders')

# Graph ko save karne ke liye (Left side-bar me save ho jayega)
plt.savefig('scheduled_shipping_days_distribution.png')
```



This cell generates a histogram plot for the 'Days for shipment (scheduled)' column to visualize its distribution. It also saves this plot as a PNG image file.

```
# 1. Sabse pehle Delivery Gap ka naya column banate hain
df['Delivery_Gap'] = df['Days for shipping (real)'] - df['Days for shipment (scheduled)']
```

```
# 2. Total orders kitne hain?
total_orders = len(df)

# KPI 1: On-Time Delivery Rate (%) -> Jo orders time par ya jaldi pahuche
on_time_orders = len(df[df['Delivery_Gap'] <= 0])
on_time_rate = (on_time_orders / total_orders) * 100
# KPI 2: Average Delivery Delay (Days) -> Jo orders late hue, unme average kitne din ka delay tha
# (Hum sirf late orders ka mean nikalenge taaki sahi delay pata chale)
avg_delay = df[df['Delivery_Gap'] > 0]['Delivery_Gap'].mean()

# KPI 3: Late Delivery Risk Ratio -> Total data me se kitne percent orders late chal rahe hain
late_orders = len(df[df['Delivery_Gap'] > 0])
late_risk_ratio = (late_orders / total_orders) * 100

# Output check karte hain
print("============ MAIN LOGISTICS KPIs ============")
print(f"1. On-Time Delivery Rate      : {on_time_rate:.2f}%")
print(f"2. Average Delivery Delay     : {avg_delay:.2f} Days")
print(f"3. Late Delivery Risk Ratio   : {late_risk_ratio:.2f}%")
print("=============================================")
```

```
============ MAIN LOGISTICS KPIs ============
1. On-Time Delivery Rate      : 42.72%
2. Average Delivery Delay     : 1.62 Days
3. Late Delivery Risk Ratio   : 57.28%
=============================================
```

This cell calculates several key performance indicators (KPIs) for logistics. It creates a 'Delivery_Gap' column, then computes the On-Time Delivery Rate, Average Delivery Delay, and Late Delivery Risk Ratio based on this gap. These KPIs quantify delivery performance.

```
print("============ 4. SHIPPING MODE EFFICIENCY INDEX ============")
# Har Shipping Mode ka average delay check karte hain
shipping_mode_index = df.groupby('Shipping Mode')['Delivery_Gap'].mean().reset_index()
print(shipping_mode_index.to_string(index=False))
print("============================================================\n")

print("============ 5. REGIONAL DELAY INDEX (TOP 5) ============")
# Top 5 regions jahan sabse zyada delay (highest average gap) chal raha hai
regional_delay_index = df.groupby('Order Region')['Delivery_Gap'].mean().sort_values(ascending=False).head(5).reset_index()
print(regional_delay_index.to_string(index=False))
print("==========================================================")
```

```
============ 4. SHIPPING MODE EFFICIENCY INDEX ============
 Shipping Mode  Delivery_Gap
   First Class      1.000000
      Same Day      0.478279
  Second Class      1.990828
Standard Class     -0.004093
============================================================

============ 5. REGIONAL DELAY INDEX (TOP 5) ============
  Order Region  Delivery_Gap
  Central Asia      0.645570
Central Africa      0.639833
    South Asia      0.597465
Western Europe      0.597403
    US Center       0.587226
==========================================================
```

This cell calculates and displays two more KPIs: 'Shipping Mode Efficiency Index' and 'Regional Delay Index'. It groups the data by shipping mode and order region, respectively, to show the average delivery gap, helping to identify modes and regions with higher delays.

model.fit(X_train, y_train): Jab LogisticRegression() chalate hain, toh yeh algorithm data ke points ko sigmoid curve ($1/(1+e^{-z})$) par fit karne ke liye math calculate karta hai.

model.predict_proba(X_test): Ye line sigmoid function ka formula lagakar data ko 0 se 100% (probability) ke beech mein badalti hai. Isme jo [:, 1] likha hai, woh late delivery hone ka risk (probability of 1) nikal kar dega

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
import numpy as np
import pandas as pd

# Check if 'df' is available from the initial data loading cell
if 'df' not in locals():
    print("Error: DataFrame 'df' not found. Please ensure the initial data loading cell (teTJUW-gW-8M) has been executed bef
else:
    # 2. Delivery_Gap aur Target Variable banana
```

```
    # 2. Delivery Gap aur Target Variable banana
    df['Delivery_Gap'] = df['Days for shipping (real)'] - df['Days for shipment (scheduled)']
    df['Late_Target'] = (df['Delivery_Gap'] > 0).astype(int)

    # 3. Text columns ko Number me badalna (Encoding)
    le_mode = LabelEncoder()
    df['Shipping Mode Enc'] = le_mode.fit_transform(df['Shipping Mode'])

    le_region = LabelEncoder()
    df['Order Region Enc'] = le_region.fit_transform(df['Order Region'])

    # 4. Features aur Data Split (Model train karne ke liye)
    X = df[['Days for shipment (scheduled)', 'Shipping Mode Enc', 'Order Region Enc']]
    y = df['Late_Target']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    # 5. Model Training
    model = LogisticRegression()
    model.fit(X_train, y_train)

    #  Poore 100% data par Risk Percentage predict karna (Sirf X_test par nahi)
    # 6. Risk Percentages predict karna (Yeh background mein sigmoid function ke weights seekhta hai)
    all_risk_scores = model.predict_proba(X)[:, 1]

    # 7. Dashboard ke liye columns ready karna
    df['Late Risk Percentage'] = (all_risk_scores * 100).round(2)

    # KPI 1: On-time Delivery Rate (Risk ka ulta)
    df['On-time Delivery Rate'] = (100 - df['Late Risk Percentage']) / 100

    # KPI 2: Avg Delivery Delay
    df['Avg Delivery Delay'] = df['Delivery_Gap'].clip(lower=0)

    # KPI 3: Late Delivery Risk Ratio (With exact 45% Threshold)
    total_orders = len(df)

    high_risk_orders = len(df[df['Late Risk Percentage'] > 45.0])
    df['Late Delivery Risk Ratio'] = high_risk_orders / total_orders # This will assign the same ratio to all rows if it's a

    # KPI 4: Shipping Mode Efficiency Index
    # Using transform('mean') will broadcast the mean back to each row based on its group
    shipping_efficiency_means = df.groupby('Shipping Mode')['Late Risk Percentage'].transform('mean')
    df['Shipping Mode Efficiency Index'] = (100 - shipping_efficiency_means) / 100

    # KPI 5: Regional Delay Index
    regional_efficiency_means = df.groupby('Order Region')['Late Risk Percentage'].transform('mean')
    df['Regional Delay Index'] = regional_efficiency_means / 100

    print("KPIs and Logistic Regression predictions added to 'df'.")
```

```
KPIs and Logistic Regression predictions added to 'df'.
```

This cell performs Logistic Regression to predict late delivery risk. It prepares features by encoding categorical columns ('Shipping Mode', 'Order Region'), splits data for training, trains a Logistic Regression model, predicts late risk percentages, and adds several new KPI columns ('On-time Delivery Rate', 'Avg Delivery Delay', 'Late Delivery Risk Ratio', 'Shipping Mode Efficiency Index', 'Regional Delay Index') to the DataFrame.

This cell handles country name standardization in `final_df`. It first checks if a cleaned country file (`cleaned_file_final.csv`) exists; if so, it loads from there. Otherwise, it uses a predefined manual mapping and Google Translate for any unmatched country names to standardize them to English, then saves the result.

```
import pandas as pd
import os # Import os for file existence check
from googletrans import Translator
import time
import unicodedata # Added for accent cleaning

output_file_city_cleaned = "cleaned_file_city_final.csv"

# Step 1: `final_df` is already loaded and contains the selected columns and KPIs, and country cleaned.
# No need to reload from CSV.

if 'final_df' not in locals():
    print("Error: DataFrame 'final_df' not found. Please ensure that 'final_df' is created in cell tf721xd0rNSG and country
else:
    if os.path.exists(output_file_city_cleaned):
        print(f"'{output_file_city_cleaned}' already exists. Loading data from it and skipping city translation.")
        # Replace final_df with the content of the pre-translated file
```

```
        final_df = pd.read_csv(output_file_city_cleaned)
        print("City standardization loaded from saved file.")
    else:
        # Step 2: Manual Spanish → English city mapping dictionary
        city_mapping = {
            "Nueva York": "New York",
            "Londres": "London",
            "París": "Paris",
            "Roma": "Rome",
            "San Pedro Sula": "San Pedro Sula",   # ✅ सिर्फ city name
            "Tegucigalpa": "Tegucigalpa",         # ✅ सिर्फ city name
            "Nal'chik": "Nalchik",
            "Tokio": "Tokyo",
            "Moscú": "Moscow",
            "Los Ángeles": "Los Angeles",
            "Ciudad de México": "Mexico City",
            "Buenos Aires": "Buenos Aires",
            "Barcelona": "Barcelona",
            "Madrid": "Madrid"
            # यहाँ आप और entries add कर सकते हैं
        }

        # Step 3: Unique values निकालें
        unique_order_cities = final_df['Order City'].unique()
        unique_customer_cities = final_df['Customer City'].unique()

        # Step 4: Unmatched detect करें
        unmatched_order_cities = [c for c in unique_order_cities if c not in city_mapping]
        unmatched_customer_cities = [c for c in unique_customer_cities if c not in city_mapping]

        # Step 5: Google Translate से auto-translate करें
        translator = Translator()
        auto_mapping = {}

        # Combine unique unmatched cities to avoid redundant translation calls
        all_unmatched_cities = list(set(unmatched_order_cities + unmatched_customer_cities))

        for city in all_unmatched_cities:
            if city not in auto_mapping:
                try:
                    translated = translator.translate(city, src='auto', dest='en').text # Use auto-detect source language
                    auto_mapping[city] = translated
                except Exception as e:
                    print(f"Error translating '{city}': {e}. Skipping and marking as original.")
                    auto_mapping[city] = city # Fallback to original city name if translation fails
                time.sleep(0.5) # Add a small delay to avoid rate limiting

        # Step 6: Final mapping merge करें
        final_mapping = {**city_mapping, **auto_mapping}

        # Step 7: Replace करें
        final_df['Order City'] = final_df['Order City'].replace(final_mapping)
        final_df['Customer City'] = final_df['Customer City'].replace(final_mapping)

        # Define the clean_names function again to ensure it's available
        def clean_names(text):
            if isinstance(text, str):
                nfkd_form = unicodedata.normalize('NFKD', text)
                return "".join([c for c in nfkd_form if not unicodedata.combining(c)])
            return text

        # Re-apply accent cleaning after translation/replacement
        final_df['Order City'] = final_df['Order City'].apply(clean_names)
        final_df['Customer City'] = final_df['Customer City'].apply(clean_names)

        # Step 8: Export cleaned CSV (intermediate export, not re-read)
        final_df.to_csv(output_file_city_cleaned, index=False)

        # Step 9: Auto-translated list ko alag CSV में save karein
        pd.Series(auto_mapping).to_csv("auto_translated_cities.csv")

        # Step 10: Summary Report
        print("📊 Summary Report")
        print("-----------------")
        print("Total unique Order Cities:", len(unique_order_cities))
        print("Manual dictionary matched (Order City):\u2002", len(unique_order_cities) - len(unmatched_order_cities))
        print("Auto-translated (Order City):\u2002", len(unmatched_order_cities))
        print("Total unique Customer Cities:", len(unique_customer_cities))
        print("Manual dictionary matched (Customer City):\u2002", len(unique_customer_cities) - len(unmatched_customer_citi
        print("Auto-translated (Customer City):\u2002", len(unmatched_customer_cities))
        print(f"\u2705 Cleaned file '{output_file_city_cleaned}' export \u0939\u094b \u0917\u0908 \u0939\u0948!")
        print("\ud83d\udcc2 Auto-translated city list saved in 'auto_translated_cities.csv'")
```

```
import pandas as pd


if 'final_df' not in locals():
    print("Error: DataFrame 'final_df' not found. Please ensure that 'final_df' is created in cell tf721xd0rNSG and city cl
else:
    print("Unique values in 'Order City' after standardization (top 20):")
    print(final_df['Order City'].unique()[:20])
    print("\nUnique values in 'Customer City' after standardization (top 20):")
    print(final_df['Customer City'].unique()[:20])

    print("\nTotal unique Order Cities after standardization:", len(final_df['Order City'].unique()))
    print("Total unique Customer Cities after standardization:", len(final_df['Customer City'].unique()))
```

```
Unique values in 'Order City' after standardization (top 20):
['Mumbai' 'San Pedro Sula' 'New York City' 'Rome' 'Munich' 'Mount Gambier'
 'Yonkers' 'Brooms' 'The Mochis' 'Tegucigalpa' 'Myrtle' 'Benin City'
 'Bayeux' 'Santiago de Cuba' 'Lagos' 'Waco' 'Kinshasa' 'Cancun'
 'Carrollton' 'Baguio City']

Unique values in 'Customer City' after standardization (top 20):
['Brownsville' 'Littleton' 'Caguas' 'Saint Mark' 'Passaic' 'Lawrence'
 'Stafford' 'Saint Anthony' 'Rivera Peak' 'Fontana' 'Taylor' 'Martinez'
 'West New York' 'North Bergen' 'Saint John' 'Peoria' 'Glenview'
 'Longview' 'Fort Worth' 'Albuquerque']

Total unique Order Cities after standardization: 3029
Total unique Customer Cities after standardization: 543
```

This cell verifies the city standardization by printing the top 20 unique values for 'Order City' and 'Customer City' from `final_df` after the cleaning process. It also reports the total number of unique cities in both columns, confirming the effectiveness of the standardization.

```
import chardet

# Detect the encoding of the CSV file
with open('/content/cleaned_file_city_final.csv', 'rb') as f:
    result = chardet.detect(f.read())

file_encoding = result['encoding']
confidence = result['confidence']

print(f"Detected encoding: {file_encoding} with confidence: {confidence:.2f}")
```

```
Detected encoding: utf-8 with confidence: 0.99
```

This cell uses the `chardet` library to detect the character encoding of the `cleaned_file_city_final.csv` file. This is crucial for correctly reading and processing the file without `UnicodeDecodeError`.

```
import pandas as pd

# `final_df` is already the most current cleaned DataFrame.
# No need to reload from CSV with encoding.

if 'final_df' not in locals():
    print("Error: DataFrame 'final_df' not found. Please ensure that 'final_df' is created in cell tf721xd0rNSG and city cl
else:
    print("Unique values in 'Order City' after standardization (top 20):")
    print(final_df['Order City'].unique()[:20])
    print("\nUnique values in 'Customer City' after standardization (top 20):")
    print(final_df['Customer City'].unique()[:20])

    print("\nTotal unique Order Cities after standardization:", len(final_df['Order City'].unique()))
    print("Total unique Customer Cities after standardization:", len(final_df['Customer City'].unique()))
```

```
Unique values in 'Order City' after standardization (top 20):
['Mumbai' 'San Pedro Sula' 'New York City' 'Rome' 'Munich' 'Mount Gambier'
 'Yonkers' 'Brooms' 'The Mochis' 'Tegucigalpa' 'Myrtle' 'Benin City'
 'Bayeux' 'Santiago de Cuba' 'Lagos' 'Waco' 'Kinshasa' 'Cancun'
 'Carrollton' 'Baguio City']

Unique values in 'Customer City' after standardization (top 20):
['Brownsville' 'Littleton' 'Caguas' 'Saint Mark' 'Passaic' 'Lawrence'
 'Stafford' 'Saint Anthony' 'Rivera Peak' 'Fontana' 'Taylor' 'Martinez'
 'West New York' 'North Bergen' 'Saint John' 'Peoria' 'Glenview'
 'Longview' 'Fort Worth' 'Albuquerque']
```

```
Total unique Order Cities after standardization: 3029
Total unique Customer Cities after standardization: 543
```

This cell is similar to `9ebe2c0a`; it re-verifies the unique cities in 'Order City' and 'Customer City' after any potential re-loading or processing, ensuring the city standardization remains consistent throughout the notebook execution.

```
print("\nChecking for 'San Luis Potosi' in 'Order City':")
print('San Luis Potosi' in final_df['Order City'].unique())

print("\nChecking for 'San Luis Potosi' in 'Customer City':")
print('San Luis Potosi' in final_df['Customer City'].unique())
```

```
Checking for 'San Luis Potosi' in 'Order City':
True

Checking for 'San Luis Potosi' in 'Customer City':
False
```

This cell specifically checks for the presence of 'San Luis Potosi' in both the 'Order City' and 'Customer City' unique value lists of `final_df`. This helps confirm if a particular city, which might have been a point of interest for the user, was correctly standardized and exists in the dataset.

```
# Check consistency between 'Order Country' and 'Order Region'
country_region_check = final_df.groupby('Order Country')['Order Region'].unique()

print("Unique Order Regions per Order Country (first 20 countries):\n")
for country, regions in country_region_check.head(20).items():
    print(f"Country: {country}, Regions: {list(regions)}")

print("\nTotal unique countries to check: ", len(country_region_check))
```

```
Unique Order Regions per Order Country (first 20 countries):

Country: Afghanistan, Regions: ['South Asia']
Country: Albania, Regions: ['Southern Europe']
Country: Algeria, Regions: ['North Africa']
Country: Angola, Regions: ['Central Africa']
Country: Arabia Saudi, Regions: ['West Asia']
Country: Argentina, Regions: ['South America']
Country: Armenia, Regions: ['West Asia']
Country: Australia, Regions: ['Oceania']
Country: Austria, Regions: ['Western Europe']
Country: Azerbaiyan, Regions: ['West Asia']
Country: Bangladesh, Regions: ['South Asia']
Country: Barbados, Regions: ['Caribbean']
Country: Belarus, Regions: ['Eastern Europe']
Country: Belgica, Regions: ['Western Europe']
Country: Belice, Regions: ['Central America']
Country: Benin, Regions: ['West Africa']
Country: Bolivia, Regions: ['South America']
Country: Bosnia and Herzegovina, Regions: ['Southern Europe']
Country: Brazil, Regions: ['South America']
Country: Bulgaria, Regions: ['Eastern Europe']

Total unique countries to check:  153
```

This cell checks for consistency between 'Order Country' and 'Order Region'. It groups the DataFrame by 'Order Country' and lists the unique 'Order Region' values associated with each country, displaying the first 20 entries to identify potential issues where a single country might be mapped to multiple regions.

```
# Identify countries with more than one unique 'Order Region'
inconsistent_countries = country_region_check[country_region_check.apply(len) > 1]

if not inconsistent_countries.empty:
    print("Countries with multiple assigned 'Order Regions' (potential inconsistencies):\n")
    for country, regions in inconsistent_countries.items():
        print(f"Country: {country}, Regions: {list(regions)}")
else:
    print("No countries found with multiple assigned 'Order Regions' in the dataset. All countries seem to have a single re
```

```
Countries with multiple assigned 'Order Regions' (potential inconsistencies):

Country: United States, Regions: ['East of USA', 'US Center ', 'South of  USA ', 'West of USA ']
```

This cell specifically identifies and prints countries that have more than one unique 'Order Region' assigned to them, indicating potential inconsistencies in the regional mapping. If no such inconsistencies are found, it confirms that all countries have a single region.

```
# Step 1: Clean up leading/trailing spaces from 'Order Region' names
final_df['Order Region'] = final_df['Order Region'].str.strip()

# Step 2: Re-check consistency between 'Order Country' and 'Order Region' after cleaning
country_region_check_cleaned = final_df.groupby('Order Country')['Order Region'].unique()
inconsistent_countries_cleaned = country_region_check_cleaned[country_region_check_cleaned.apply(len) > 1]

print("\n--- Order Region Consistency Check After Stripping Whitespace ---")
if not inconsistent_countries_cleaned.empty:
    print("Countries with multiple assigned 'Order Regions' after cleaning (potential inconsistencies):\n")
    for country, regions in inconsistent_countries_cleaned.items():
        print(f"Country: {country}, Regions: {list(regions)}")
else:
    print("No countries found with multiple assigned 'Order Regions' after cleaning. All countries seem to have a single re

# Step 3: Display the unique regions for 'United States' specifically after stripping
print("\nUnique regions for 'United States' after stripping:")
print(final_df[final_df['Order Country'] == 'United States']['Order Region'].unique())
```

```
--- Order Region Consistency Check After Stripping Whitespace ---
Countries with multiple assigned 'Order Regions' after cleaning (potential inconsistencies):

Country: United States, Regions: ['East of USA', 'US Center', 'South of  USA', 'West of USA']

Unique regions for 'United States' after stripping:
['East of USA' 'US Center' 'South of  USA' 'West of USA']
/tmp/ipykernel_28096/1504674436.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  final_df['Order Region'] = final_df['Order Region'].str.strip()
```

This cell refines the 'Order Region' consistency by first stripping any leading or trailing whitespace from the 'Order Region' column. It then re-checks for countries with multiple assigned regions, specifically displaying the regions for 'United States' after this cleaning step.

```
# Step 1: Create a mapping for the United States regions to a single standardized region
region_standardization_map = {
    'East of USA': 'North America',
    'US Center': 'North America',
    'South of  USA': 'North America',
    'West of USA': 'North America'
}

# Step 2: Apply the mapping to the 'Order Region' column
final_df['Order Region'] = final_df['Order Region'].replace(region_standardization_map)

# Step 3: Re-check consistency between 'Order Country' and 'Order Region' after standardization
country_region_check_final = final_df.groupby('Order Country')['Order Region'].unique()
inconsistent_countries_final = country_region_check_final[country_region_check_final.apply(len) > 1]

print("\n--- Order Region Consistency Check After Full Standardization ---")
if not inconsistent_countries_final.empty:
    print("Countries with multiple assigned 'Order Regions' after full standardization (potential inconsistencies):\n")
    for country, regions in inconsistent_countries_final.items():
        print(f"Country: {country}, Regions: {list(regions)}")
else:
    print("No countries found with multiple assigned 'Order Regions' after full standardization. All countries now seem to h

# Display the unique regions for 'United States' specifically after full standardization
print("\nUnique regions for 'United States' after full standardization:")
print(final_df[final_df['Order Country'] == 'United States']['Order Region'].unique())
```

```
--- Order Region Consistency Check After Full Standardization ---
No countries found with multiple assigned 'Order Regions' after full standardization. All countries now seem to have a singl

Unique regions for 'United States' after full standardization:
['North America']
/tmp/ipykernel_28096/2462506885.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  final_df['Order Region'] = final_df['Order Region'].replace(region_standardization_map)
```

This cell standardizes the 'Order Region' values for 'United States' by mapping specific regional entries (e.g., 'East of USA', 'US Center') to a single 'North America'. It then re-verifies that after this full standardization, no country is associated with multiple regions.

Removing unnecessary columns

To predict late delivery risk and avoid penalties, I am dropping the columns `Order Item Total`, `Order Item Discount`, and `Order Item Quantity` from the `final_df`. This focuses the dataset on the most relevant features for the predictive model.

```
columns_to_drop = ['Order Item Total', 'Order Item Discount', 'Order Item Quantity']
final_df = final_df.drop(columns=columns_to_drop, errors='ignore')

print(f"Dropped columns: {columns_to_drop}")
print("Current columns in final_df:")
print(final_df.columns.tolist())

display(final_df.head())
```

```
Dropped columns: ['Order Item Total', 'Order Item Discount', 'Order Item Quantity']
Current columns in final_df:
['Days for shipping (real)', 'Days for shipment (scheduled)', 'Late_delivery_risk', 'Delivery Status', 'Shipping Mode', 'Sal
```

	Days for shipping (real)	Days for shipment (scheduled)	Late_delivery_risk	Delivery Status	Shipping Mode	Sales	Order Profit Per Order	Order Region	Order Country	Order City	...	Category Name	D
0	6	4	1.0	Late delivery	Standard Class	499.95	159.69	South Asia	India	Mumbai	...	Cardio Equipment	
1	4	4	0.0	Shipping on time	Standard Class	199.95	48.71	Central America	Honduras	San Pedro Sula	...	Shop By Sport	
2	4	4	0.0	Shipping on time	Standard Class	199.99	87.36	Central America	Honduras	San Pedro Sula	...	Water Sports	
3	6	4	1.0	Late delivery	Standard Class	199.99	-41.89	North America	United States	New York City	...	Water Sports	
4	6	4	1.0	Late delivery	Standard Class	50.00	10.00	North America	United States	New York City	...	Women's Apparel	

5 rows × 24 columns

This cell drops specified columns ('Order Item Total', 'Order Item Discount', 'Order Item Quantity') from `final_df` as they were deemed less relevant for late delivery risk prediction. It then prints the updated list of columns and displays the head of the modified DataFrame.

Applying Standardization to 'Market' Column

To enhance clarity and consistency, I will standardize the 'Market' column by replacing abbreviations with their full geographical names.

```
market_standardization_map = {
    'LATAM': 'Latin America',
    'USCA': 'United States & Canada'
}

final_df['Market'] = final_df['Market'].replace(market_standardization_map)

# Fill NaN values in 'Market' with 'Unknown'
final_df['Market'] = final_df['Market'].fillna('Unknown')

print("Unique values in 'Market' column after standardization:")
print(final_df['Market'].unique())
print(f"\nTotal unique markets: {len(final_df['Market'].unique())}")

display(final_df.head())
```

```
Unique values in 'Market' column after standardization:
['Pacific Asia' 'Latin America' 'United States & Canada' 'Europe' 'Africa']

Total unique markets: 5
```

	Order Customer Id	Days for shipping (real)	Days for shipment (scheduled)	Late_delivery_risk	Delivery Status	Shipping Mode	Sales	Order Profit Per Order	Order Region	Order Country	...	Customer Country
0	1	6	4	1	Late delivery	Standard Class	499.95	159.69	South Asia	India	...	EE. UU.
1	2	4	4	0	Shipping on time	Standard Class	199.95	48.71	Central America	Honduras	...	EE. UU.
2	2	4	4	0	Shipping on time	Standard Class	199.99	87.36	Central America	Honduras	...	EE. UU.
3	2	6	4	1	Late delivery	Standard Class	199.99	-41.89	East of USA	United States	...	EE. UU.
4	2	6	4	1	Late delivery	Standard Class	50.00	10.00	East of USA	United States	...	EE. UU.

5 rows × 24 columns

This cell standardizes the 'Market' column by replacing abbreviations like 'LATAM' with 'Latin America' and 'USCA' with 'United States & Canada'. It then prints the unique values after standardization and displays the head of the DataFrame to show the changes.

Verifying 'Market' Column Standardization

```
display(final_df[['Order Region', 'Market', 'Customer Segment']].head())
```

	Order Region	Market	Customer Segment
0	South Asia	Pacific Asia	Consumer
1	Central America	Latin America	Consumer
2	Central America	Latin America	Consumer
3	North America	United States & Canada	Consumer
4	North America	United States & Canada	Consumer

This cell displays the first few rows of the 'Order Region', 'Market', and 'Customer Segment' columns from `final_df`. This is a quick visual check to verify that the standardization applied to the 'Market' and 'Order Region' columns is correctly reflected in the DataFrame.